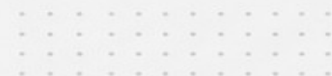


The Tokenomics of Data-Serialization Formats

A cross-provider engineering reference to serialization-format efficiency, tokenizer behavior, pricing, and reliability.

June 2026





The executive question

Serialization looks like syntax, but in AI systems it becomes unit economics: every repeated key, delimiter, newline, tag, and repair loop becomes billed context.

COST PRESSURE

63%

CSV used 62.9% fewer tokens than pretty JSON in the flat TOON benchmark.

PRICING LEVERAGE

5–6×

Output tokens cost five to six times input on representative flagship models.

BENCHMARK SCOPE

12

Figure 2 uses a complete 12-format, internally consistent benchmark setup.

CORE WARNING

Shape

The right format depends on data shape, usage context, and validation requirements.

Decision target: minimize total billed tokens without sacrificing comprehension accuracy, parseability, or operating discipline.



Three evidence classes, one guardrail

The report separates token-efficiency measurement, comprehension accuracy, and provider pricing so the deck does not mix incompatible benchmarks.

01

TOON token efficiency

100 uniform employee records, measured with GPT-5 o200k_base tokenizer.

02

ImprovingAgents

1,000 records, 1,000 randomized retrieval questions, GPT-4.1-nano, 12 paired format results.

03

Provider pricing

Official list prices verified June 16, 2026, expressed per million tokens.

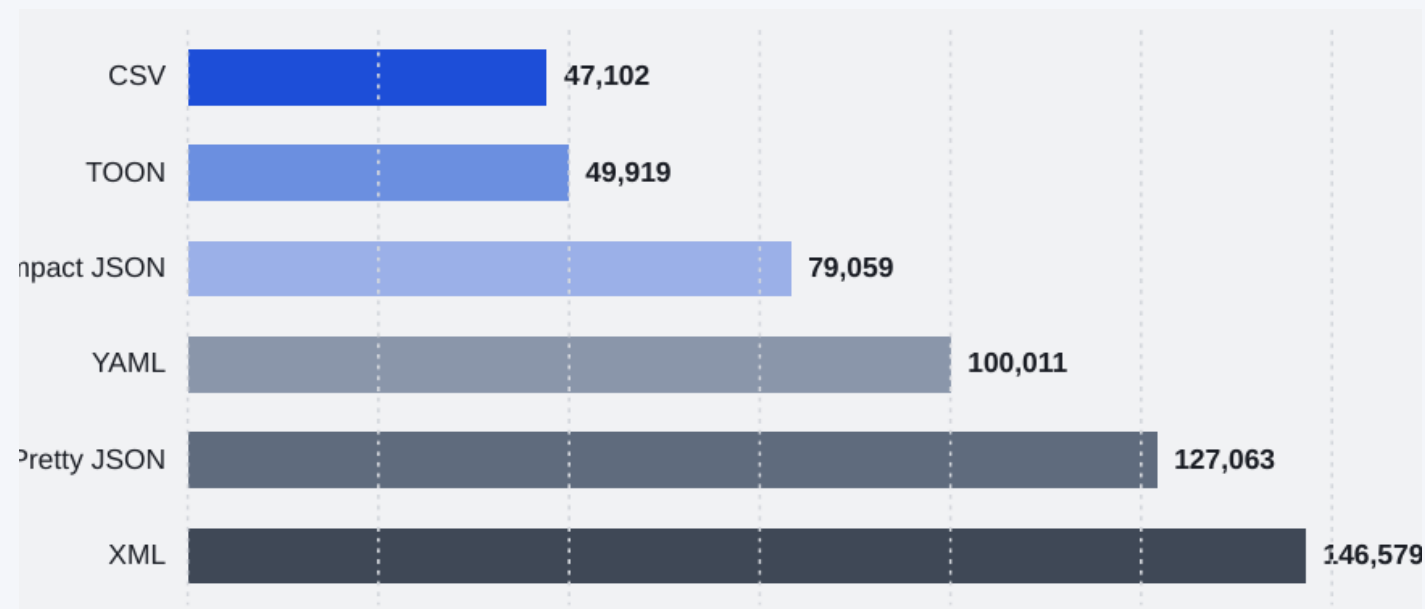
Guardrail: do not combine token counts and accuracy from different datasets, tokenizer families, or serialization settings in the same scatter plot.



Bottom line for production teams

- | | | |
|-----------|--------------------------------------|---|
| 01 | Flat, high-volume input | Use CSV or TSV as the cost floor. Evaluate TOON when uniform arrays justify schema-once encoding. |
| <hr/> | | |
| 02 | Nested or heterogeneous input | Use compact JSON as the safest efficiency baseline; TOON can lose on mixed or deeply nested structures. |
| <hr/> | | |
| 03 | Structured output | Use JSON Schema or strict tool calling when validation matters. |
| <hr/> | | |
| 04 | Benchmark discipline | Measure target model, tokenizer/count endpoint, serializer configuration, parse failures, retries, and total billed tokens. |
| <hr/> | | |

Input cost floor: CSV leads flat tables



Key reading

CSV is the least-expensive representation in this flat-data case.

TOON is 6.0% larger than CSV, but 36.9% smaller than compact JSON.

XML is the most expensive because tag names repeat in opening and closing tags.

Production implication: first remove removable whitespace and repeated fields before considering a new serialization grammar.

Shape drives the format choice

FORMAT FAMILY	MAIN OVERHEAD	TYPICAL TOKEN POSITION	BEST FIT
CSV / TSV	Header once; delimiters and newlines	Lowest for flat tables	Flat, uniform rows
TOON	Schema-once tabular encoding	Very low on uniform arrays; can lose to compact JSON on mixed/nested data	Large, repeated uniform input
Markdown table / KV	Pipes or repeated key-value labels	Low to medium	Reasoning-heavy prompts and human review
Compact JSON	Repeated keys, no optional whitespace	Medium	Robust default and machine interchange
YAML	Indentation and dash/colon markers	Medium; result depends on JSON baseline	Human-authored config
Pretty JSON	Repeated keys plus indentation	High	Debugging and inspection
XML	Opening and closing tags	Highest in the cited flat benchmark	Legacy integration and document markup

The mistake to avoid

A format can be excellent for uniform arrays and still be weaker for mixed structures. The report notes TOON can become 14.7% larger than compact JSON across a mixed-structure track.

DEFAULT

Compact JSON

Best baseline for nested, heterogeneous, API-facing data.

EXCEPTION

CSV / TOON

Strongest only when the data shape is flat or uniform enough.

Cheapest is not always best



What changes the decision

Markdown-KV leads accuracy at 60.7%, but uses 52,104 tokens.

CSV is cheapest at 19,524 tokens, but scored 44.3%.

Markdown table is the strongest compact compromise in this benchmark.

This is not a universal ranking; it is one model, one synthetic dataset, and one retrieval task.

Complete 12-format result set

#	FORMAT	ACCURACY	TOKENS	RANK
1	Markdown-KV	60.7%	52,104	1
2	XML	56%	76,114	2
3	INI	55.7%	48,100	3
4	YAML	54.7%	55,395	4
5	HTML	53.6%	75,204	5
6	JSON	52.3%	66,396	6
7	Markdown table	51.9%	25,140	7
8	Natural language	49.6%	43,411	8
9	TOON	47.5%	21,518	9
10	JSONL	45%	54,407	10
11	CSV	44.3%	19,524	11
12	Pipe-delimited	41.1%	43,098	12

Interpretation

The 12-format table completes the corrected publication scope: TOON is included alongside every format from the ImprovingAgents run.

Accuracy winner: Markdown-KV at 60.7%.

Cheapest format: CSV at 19,524 tokens.

Compact compromise: Markdown table at 51.9% and 25,140 tokens.

TOON reached 47.5% at 21,518 tokens, with no statistically significant difference from CSV in that benchmark.

Same record, every format



CSV 23.3 tok/rec · 2,334 tot [M] <pre>id,name,age,city,department,salary,years_experience,project_count 1,Diana A0,46,London,Engineering,141015,7,17</pre>	TOON 25.0 tok/rec · 2,498 tot [M] <pre>employees[100] {id,name,age,city,department,salary,years_experience,project_count}: 1,Diana A0,46,London,Engineering,141015,7,17</pre>
Compact JSON 39.2 tok/rec · 3,924 tot [M] <pre>[{"id":1,"name":"Diana A0","age":46,"city":"London","department":"Engineering","salary":141015,"years_experience":7,"project_count":17}]</pre>	YAML 49.6 tok/rec · 4,959 tot [M] <pre>employees: - id: 1 name: Diana A0 age: 46 city: London department: Engineering salary: 141015 years_experience: 7 project_count: 17</pre>
Pretty JSON 63.3 tok/rec · 6,331 tot [M] <pre>[{ "id": 1, "name": "Diana A0", "age": 46, "city": "London", "salary": 141015 }]</pre>	XML 73.0 tok/rec · 7,296 tot [M] <pre><employees> <employee id="1"> <name>Diana A0</name> <age>46</age> <city>London</city> <salary>141015</salary> </employee> </employees></pre>

Why the count changes

CSV pays for the header once, then compact row values.

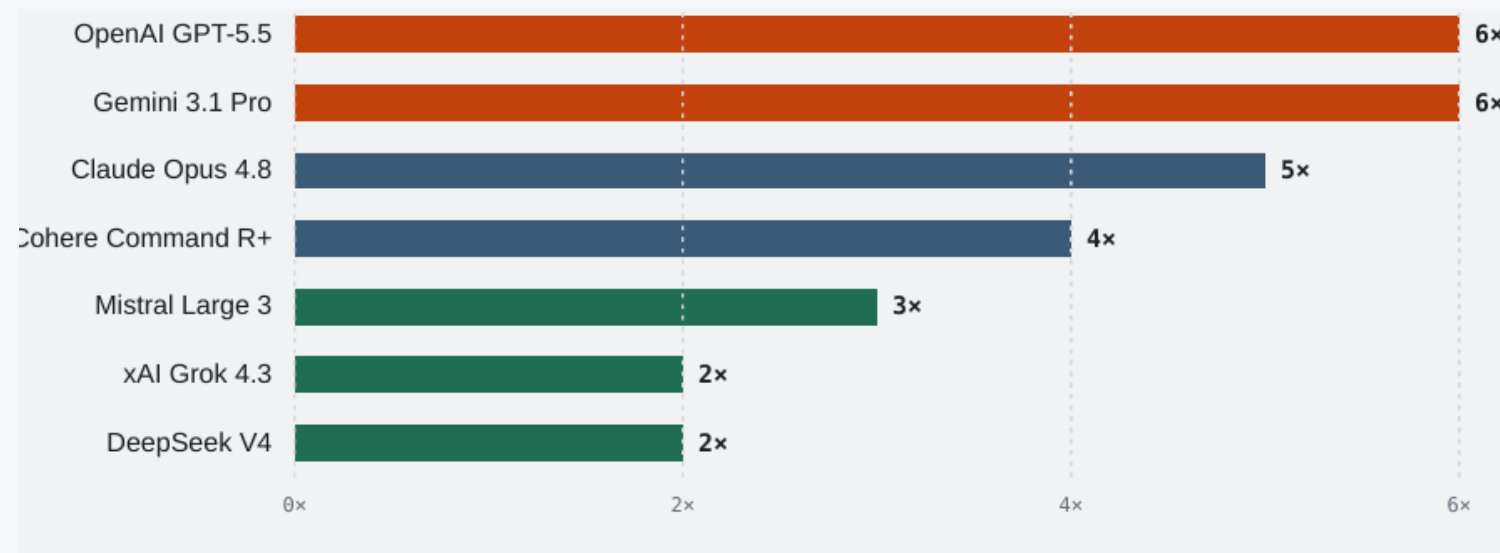
Compact JSON repeats keys on every object.

Pretty JSON adds removable whitespace.

XML repeats semantic labels twice: opening and closing tags.

This is the intuition behind the aggregate chart.

Output tokens dominate savings math



Production reading

OpenAI GPT-5.5 and Gemini 3.1 Pro price output at 6× input.

Claude Opus 4.8 is 5× input.

Grok 4.3 and DeepSeek V4 sit at 2×, so equal token cuts save less.

Optimization priority: compress high-volume input first, but stabilize structured output before chasing novel output formats.

Name the baseline before claiming savings

FIGURE 4. YAML TOKEN DELTA AGAINST TWO JSON BASELINES



The phrase "YAML is more token-efficient than JSON" is incomplete until the JSON baseline is named. Within one TOON 100-employee dataset, YAML is 21.3% smaller than pretty JSON but 26.5% larger than compact JSON. Optional JSON whitespace can be removed; YAML indentation and line structure are part of its syntax. For automated prompt assembly, compact JSON is the relevant baseline.

MONTHLY TABULAR INPUT	PRETTY JSON	CSV ESTIMATE	ESTIMATED SAVING
10M tokens	\$30.00	\$11.12	\$18.8
100M tokens	\$300.00	\$111.21	\$188.7
1B tokens	\$3,000.00	\$1,112.09	\$1,887.9



This is not a promise of production savings. It assumes the workload has the same flat-data shape and token ratio as the benchmark, excludes caching and batch discounts, and treats the stated token volume as the pretty-JSON baseline.

YAML is 21.3% smaller than pretty JSON but 26.5% larger than compact JSON in the same TOON 100-employee dataset. For automated prompt assembly, compact JSON is the relevant baseline.



Savings only count if the system can trust the result

Token reduction is economically useful only if the downstream system can parse the result and avoid repair loops.

OUTPUT METHOD	TOKEN EFFICIENCY	VALIDATION STRENGTH	OPERATIONAL RISK
Strict JSON Schema / tool call	Medium	Highest	Lowest for machine consumption
Plain compact JSON + validator	Medium	High with retries	Moderate
CSV / TSV + parser + adversarial tests	High for flat output	Medium	Delimiter, quote, newline failures
YAML / TOON generated from prompting	Workload-dependent	Low to medium	Prompt tax, syntax repair, unfamiliarity

Output rule

Use provider-native JSON Schema or strict tool/function calling for machine-consumed outputs.

Plain compact JSON plus validator is acceptable with retries.

Generated CSV, YAML, or TOON requires custom escaping, validation, and repair logic.

A practical selection logic



Is the payload flat and tabular?	Use CSV or TSV for input; validate escaping and embedded newlines.
Is it a large, uniform array of objects?	Benchmark TOON against CSV and compact JSON using the target model.
Is the structure nested, mixed, or API-facing?	Use compact JSON as the default interchange format.
Is the model generating machine-consumed output?	Use JSON Schema structured output or strict tool calling.
Is human editing the dominant requirement?	Use YAML/TOML for maintainability — not on a blanket token claim.

Use this order

Flat and tabular → CSV or TSV.

Large uniform array → benchmark TOON against CSV and compact JSON.

Nested or mixed → compact JSON.

Machine output → JSON Schema or strict tool call.

Human-authored config → YAML/TOML for maintainability, not blanket token savings.



Implementation next steps

- 01 Adopt compact JSON as the machine-interchange default.
- 02 Use CSV/TSV for validated flat-table inputs.
- 03 Benchmark TOON only for large uniform object arrays.
- 04 Lock structured-output baselines with JSON Schema or strict tool calls.
- 05 **Record serializer version, tokenizer/count endpoint, payload hash, model, price date, parse failures, retries, latency, and total billed tokens.**

Close: optimize notation only after data selection, benchmark discipline, and output reliability are already controlled.



Repository

github.com/emblem-nlp